

React Core Concepts – Day 1 & Day 2 (Why, What, How)

This document explains React fundamentals with clear WHAT, WHY, and HOW answers, interview-oriented explanations, and practical examples.

■ Day 1 – React Basics & Virtual DOM

1. What is React, and why was it created?

What: React is a JavaScript library for building user interfaces, mainly single-page applications.

Why: It was created by Facebook to efficiently update complex UIs where data changes frequently.

How: React uses components, state, props, and a virtual representation of the DOM to update UI efficiently.

Why library, not framework?

React focuses only on UI rendering. Routing, state management, and HTTP are handled by external libraries, unlike frameworks that provide everything.

React vs jQuery / Vanilla JS

In jQuery, developers manually manipulate the DOM. In React, developers describe UI using state, and React updates the DOM automatically.

2. What is the Virtual DOM?

What: A lightweight JavaScript object that represents the real DOM.

Why: Direct DOM manipulation is slow.

How: React compares old and new Virtual DOM trees and updates only the changed parts.

Is Virtual DOM always faster?

No. It adds overhead for small apps, but scales very efficiently for large, dynamic UIs.

3. What is Reconciliation?

Reconciliation is the process of comparing old and new Virtual DOM trees to decide minimal UI updates.

Keys: Keys help React identify which items changed, were added, or removed in lists.

4. How React updates UI after state change

State update → New Virtual DOM → Diffing → Commit changes to real DOM.

Fiber: Allows React to pause, prioritize, and resume rendering work for better performance.

Mutating state directly prevents React from detecting changes, causing bugs.

5. Output-based Example

The `console.log` runs every time the component renders, including parent re-renders.

■ Day 2 – Components & Rendering

1. Functional vs Class Components

Functional components are plain JavaScript functions. Class components use ES6 classes.

Hooks made functional components powerful by enabling state and lifecycle features.

2. Props vs State

Props are read-only data passed from parent. State is local, mutable data inside a component.

Both prop change and state change trigger re-render.

3. When does React re-render?

On state update, prop update, or parent re-render.

Setting same state value usually does not cause re-render due to `Object.is` comparison.

`React.memo` prevents unnecessary re-renders by memoizing components.

4. Component Lifecycle

Mount → Update → Unmount

`useEffect` replaces `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.

`Cleanup` prevents memory leaks like event listeners.

5. Counter Output Example

Rendered logs once initially + once per state change.

Calling `setCount(count)` does not trigger re-render because value remains same.